ForgeFlow: A Human-in-the-Loop AI Platform for End-to-End Web Application Delivery with a Bounded, Diff-Aware Self-Healing Repair Loop

Faheemuddin Sayyed
Department of CSE
Symbiosis Institute of Technology,
Symbiosis Ineternational University,
Pune, India
faheemuddinsayyed789@gmail.com

Sanidhya Awasthi
Department of CSE
Symbiosis Institute of Technology,
Symbiosis International University,
Pune, India
sanidhya.awasthi19@gmail.com

Raghav Sonchhatra
Department of CSE
Symbiosis Institute of Technology,
Symbiosis International University,
Pune, India
rsonchhatra49@gmail.com

Dr. Payal Kadam
Department of CSE
Symbiosis Insitute of Technology,
Symbiosis International University,
Pune, India
payal.kadam@sitpune.edu.in

***Abstract*—Creating a web application is almost trivial with large-language-model coding assistants and "prompt-to-app" builders, but verifying it is challenging. Empirical evidence from recent years demonstrates that AI-generated artifacts are systematically flawed in maintainability and security, that repeatedly asking the model to refine its own code can lead to a decrease in its security, that the success rate of the first deployment of the infrastructure generated by the model is less than a third, and that the unlimited deployment of the model leads to the consumption of budget long after the marginal gain has been lost. This paper introduces ForgeFlow, an AI-native, human-in-the-loop Progressive Web Application that takes an idea for a web-app (provided as screenshots or as a structured specification) and carries it through the process of requirements capture, UI design, generation of full stack code, generation of requirements-based tests, bounded, diff-aware self-healing repair loop, infrastructure-aware dual-mode cloud deployment, and validation against the live URL. ForgeFlow is different from research agents and commercial builders through two design commitments. First, the repair loop is explicitly terminating: it is given a small context, just the failing tests, the sources on which the tests are exercised, and the Git diff since the last passing run, and the loop stops when the first of a regression guard, a no-progress guard, an iteration cap or a budget cap is reached, and the human is handed an auditable "here is where I am stuck" summary. Second, non-linearity is not a UI affordance; each stage preserves its own state, there is no requirement that any particular stage be present, and only two of the stages are ordered. We explain the architecture, the traceability chain that connects acceptance criteria to executable tests and back to repair context, and a full prototype of approximately 41 000 lines of production code and 1 491 automated tests, and the evaluation protocol that measures the convergence of repairs.**

Keywords*—AI-native software engineering, large language model agents, automated program repair, self-healing software, human-in-the-loop systems, automated test generation, requirements traceability, DevOps automation, progressive web application

I. Introduction

There are now a number of large language models that are used throughout the entire software lifecycle, with a recent survey systemizing 926 studies on 112 software engineering tasks throughout 5 phases of the SE workflow [1]. Concurrently, a user-centric category has grown up in which a natural-language query is directly converted to a deployable application, a survey of vibe coding in 2025 identifies and formalizes existing practice, including planning-driven and test-driven forms of vibe coding [2]. Adoption is no longer fringe: The 2025 Stack Overflow Developer Survey shows that the number of developers using or intending to use AI tooling has dropped from about 84% to 76% year on year [3] and that trust in the accuracy of AI output has declined year on year.

This is not a perception gap, it's a measurable one in the artifacts themselves. In 2026, researchers conducted a study on the mining of build code generated by AI across the three most popular build systems: Maven, Gradle, CMake, and Make, and identified a set of maintainability and security smells that agents systematically introduced in the build code [4]. A more direct application to any system that is able to repair its own output is a controlled study of repeatedly asking a model to generate more secure code, which showed that the security properties of the code could monotonically weaken with repeated iterations rather than monotonically strengthen, and an unbounded "keep fixing it" loop is not just a waste but it is actually dangerous [5].

The weakness is most apparent during deployment. The success rates for two frontier models were 30.2 % and 26.8 % to deploy them successfully the first time, when tested on a benchmark of real tasks [6]. Generation and deployment are therefore not the same issue and passing a local test suite is not proof of behaviour.

The second axis of failure is control. A large scale study of 20 574 real-world coding-agent sessions identifies pervasive developer–agent misalignment, where the agent's actions differ

from what the developer intended [7]. A complementary study of professional practice revealed that experienced developers are not giving control over agents to them, but constantly restraining, monitoring and directing them [8]. It is therefore a requirement for any system that is intended to be trusted to be inspectable, revisitable and skippable: even by the users who can't read the code it generates.

Last but not least, the repair loop has an economics. A 2026 empirical study of LLM-based repair loops investigates the number of iterations that are worth performing, and a follow-up study over model sizes and benchmarks reveals that the vast majority of the benefits of the repair loops are gained after 1-3 iterations, with subsequent iterations consuming tokens for the most part [9, 10]. In isolation, LLM test generation fails badly when the program under test changes, with most of the new test failures still passing on the original program (the model tests the code it remembers, not the code it was provided) and the authors explicitly recommend computing a code diff and marking the changes to guide the model's attention [11].

These five findings are the impetus to ForgeFlow. Its contributions are:

1) A bounded, diff-aware self-healing repair loop that is only fed failing tests, the sources those tests exercise, the Git diff since the last passing run, and the acceptance-criterion text; and that terminates on the first of a regression guard, a no-progress guard, an iteration cap or a budget cap.

2) Escalation to first class terminal state. Non-convergence is a normal outcome that is recorded and provides the human with a structured summary of the failed tests, patches tried and convergence metrics, not an error or a silent loop.

3) Non-linearity as a principle of the architecture. The six stages are independent and a project can be entered at any stage, any stage can be skipped, refinement means that work downstream is stale but not deleted. Only deploy ⇐ build and validate ⇐ deploy is enforced.

4) A requirement-to-repair traceability chain consisting of a single criterion identifier that passes through the following steps: User authored acceptance criterion, test generation, test execution, failure analysis, repair context.

5) Infrastructure-aware, dual-mode deployment and post-deployment validation, where the generated suite is re-executed against the live URL and live failures re-enter the same bounded repair loop.

Section II summarizes the literature for the five threads for the 2025–2026 period and identifies a research gap. The non-linear stage model and the architecture are described in Section III. The requirements-driven verification is covered in Section IV. The repair loop is given in Section V. Section VI is on deployment and live validation. The implementation is reported in Section VII. The evaluation protocol, threats to validity and limitations are presented in Section VIII. Section IX concludes.

II. Related Work

A. Agentic Code Generation and Issue Resolution:

Agentic software issue resolution has developed into a subfield, which can be categorized into benchmarks, techniques and empirical studies, based on over 150 main studies, in a 2025 survey [12]. A major counter-current is Agentless, which showed that a simple, structured three-phase pipeline could perform at least as well as complex autonomous agents on issue resolution and use significantly less budget [13]; a result that suggests that simple, structured scaffolding can beat unconstrained agency. Meanwhile, fully autonomous repair agents like RepairAgent that can dynamically plan and invoke repair tools using an LLM have been demonstrated to successfully repair real-world defects at competitive rates [14]. The Agentless lesson is the lesson ForgeFlow uses, as opposed to the autonomous-agent lesson, where the pipeline is fixed and deterministic, an opinionated project skeleton is instantiated by file copy instead of generated, and the model is invoked only when actual synthesis is needed.

B. Program Repair and the Economics of Iteration:

The results of these fine-tuning studies, published in IEEE TSE, prove that the quality of the repairs generated by an LLM is highly dependent on the way the repair task is formulated and the context provided [15]. The 2026 repair-loop literature brings the economic dimension to the earlier literature missing: how many attempts are worth it [9] and where is the marginal return at the end [10] across model scales and benchmarks. The security-degradation analysis of iterative generation provides the safety argument for the same conclusion, that later generations are not just low-yield, but potentially harmful [5]. These loops are measured by existing work; they are not normally designed into a delivery system. ForgeFlow implements iteration caps, stall detection, a regression guard and a budget guard as first-class, configurable, auditable controller state.

C. Test Generation and Requirements Traceability:

Automated Test Generation using LLMs is well-defined. A large-scale study in Empirical Software Engineering was performed to evaluate the effectiveness of prompt-engineering strategies for producing class-level unit-tests over hundreds of thousands of produced tests [16] and an ISSTA study fine-tuned 37 models across three different unit-testing scenarios and five benchmarks [17]. Feature-driven end-to-end test generation [18] and hybrid program-analysis-guided iterative generation [19] are closer to ForgeFlow's goal: generate executable E2E tests from application features and enhance the validity of generated suites. On the requirements side, prompt-engineered LLM traceability has been demonstrated to be able to effectively recover requirement-to-artifact links [20]. The critical negative result is [11] generated suites lose validity when the code changes, and the recommended mitigation is a highlighted diff. The minimal repair context is exactly a diff-highlighted context and the context was designed and implemented for the same reason.

D. Human-in-the-Loop Control

It's not about autonomy, it's about human-in-the-loop, which is what is used in industry. HULA, deployed within Atlassian Jira, maintains engineers' control over both the coding plan and the resulting code, and the follow-up reports on meaningful reductions in development effort [21] with the cost of correctness testing and the variability of LLM-based evaluation as the main open challenges [22]. The professionals' deliberate control over the process, not delegating everything, is confirmed in independent observational work; what happens when the professionals are unable to do this is quantified in large-scale session analysis [7, 8]. ForgeFlow abstracts this from "review the plan and the patch" to "enter, refine, skip or revisit any stage of the lifecycle", and it considers the cost of correctness testing mentioned in [22] as a first-class engineering constraint, by model routing and budget caps.

E. Multimodal Design-to-Code

Design2Code benchmarks multimodal code generation from 484 real web pages, and demonstrates that models are still behind when it comes to visual element recall and layout fidelity [23]. But ForgeFlow is not just a one-shot conversion whose output is blindly fed downstream, it's a pluggable, human-refinable stage that has versioned artifacts and an explicit skip path.

F. Research Gap

Table I summarises the gap. Research systems don't deploy and don't validate the deployed artifact; commercial builders deploy but don't validate; no one validates, and no one escalates for non-convergence.

TABLE I. CAPABILITY COMPARISON ACROSS SYSTEM CLASSES

Capability	Research agents [12]–[14]	LLM test/repair tools [15]–[19]	Commercial builders [2]	ForgeFlow
Multimodal design intake (screenshot → UI)	✗	✗	✓	✓ (pluggable)
Full-stack application generation	partial	✗	✓	✓ (fixed stack)
Requirements → executable tests (traceable)	✗	partial	✗	✓ (structured)
Bounded repair iteration cap + stall detection	✗	✗	✗	✓
Diff-aware minimal repair context	✗	partial	✗	✓
Human escalation on non-convergence	✗	✗	✗	✓
Automated cloud deployment	✗	✗	✓ (locked-in)	✓ (dual-mode)
Post-deployment live validation	✗	✗	✗	✓
Non-linear, skip-any-stage control	✗	✗	partial	✓
Per-run cost accounting and budget caps	✗	✗	✗	✓

III. SYSTEM DESIGN

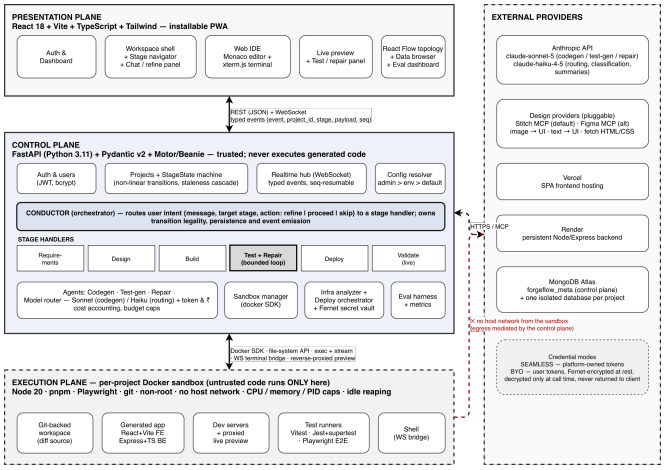


Fig. 1. ForgeFlow system architecture: the presentation, control and execution planes, and the external provider boundary. Untrusted generated code executes only inside the per-project sandbox, which has no host network.

A. Design Principles

P1: Verification over generation. The defensible contribution is knowing when the system is finished, not how quickly it emits code.

P2: One fixed generated-app stack. Every generated application is React (Vite + TypeScript) on the frontend, Node/Express (TypeScript) on the backend, and MongoDB via Mongoose, tested with Vitest, Jest + supertest, and Playwright. Constraining the target raises first-pass generation success, reduces the number of repair iterations required, and makes rigorous verification tractable within a student project's budget. It also lets a hand-written 44-file skeleton be *copied* rather than generated, which removes scaffolding tokens entirely — the deterministic-scaffolding lesson of [13].

P3: Cost discipline as an architectural concern. Two model roles are separated: a code-generation model (claude-sonnet-5) for synthesis, repair and test generation, and a cheap routing model (claude-haiku-4-5) for classification, summarisation and the freeform-description-to-feature-list

draft. Every call records token usage against the project, and per-project and global budget caps halt work with an actionable message. This is a direct engineering answer to the "high cost of correctness testing" challenge named in [22]. A provider abstraction additionally allows any OpenAI-compatible endpoint to be substituted without touching agent, tool or cost code.

P4: Never overwrite; always version. Artifacts — designs, code changes, requirements, test suites, test runs, repair attempts, deployments, infrastructure plans — are versioned, never replaced, which is what makes "revisit and refine" safe.

P5: Untrusted code runs only in sandboxes. The control plane never executes generated code. Each project receives a Docker container running as a non-root user with no host network, dropped capabilities, and CPU, memory and PID limits, reaped when idle. Recent work on securing test environments for untrusted code documents how wide the escape surface of naïvely configured execution environments actually is [24]; ForgeFlow treats the sandbox boundary, not the model's good behaviour, as the security control.

P6: Non-linearity is an invariant, not a feature.

B. Architecture

The structure of ForgeFlow is divided into three planes and an external provider boundary (Fig. 1). The control plane is a FastAPI (Python 3.11) service that has Pydantic v2 schemas at all boundaries and Motor/Beanie on top of MongoDB. It includes authentication, project lifecycle and state of the project stages, the conductor, the six stage handlers, the agent layer, the sandbox manager, the deployment orchestrator, the encrypted credential vault, the evaluation harness, and a WebSocket hub. For every project a separate application database is created, and there is a forgeflow_meta database for control plane metadata.

Each execution plane consists of a single Docker container that contains the workspace backed by Git, the generated application, the reverse-proxied preview of the application's development servers, and the test runners. The diff in Section V is not just a side-benefit of Git backing: It is what makes it computable.

The presentation plane is a PWA (Progressive Web App) installable using React 18: dashboard, a clickable stage navigator (workspace shell), a Monaco-based IDE that renders code as the agent writes it, an xterm.js terminal bridged to the sandbox shell, a live preview pane, a test-and-repair pane, a React Flow deployment topology, and a project data browser.

All work of long duration is event driven. A typed event has {event, project_id, stage, payload, seq}; clients subscribe per project and resume from the previous seq after disconnect, so there is no loss of an in-flight generation/repair loop when the page is reloaded**.**

C. The Non-Linear Stage Model

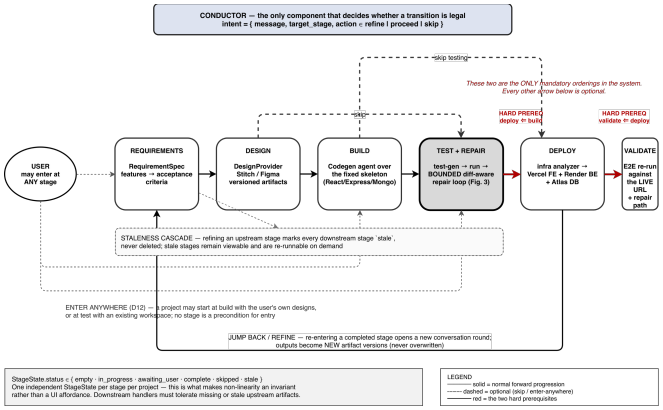


Fig. 2. The non-linear stage model is a model that assumes the stage is non-linear. Dashed edges are not required (enter-anywhere and skip); the two bold edges are the only required orderings; artifacts in downstream edges are not deleted, but a staleness cascade is propagated.

The flow of the product is requirements – design – build – test – deploy – validate. This order pushes the navigator, the default entry stage and the staleness cascade and gates nothing. One StageState is maintained for each project at each stage, and has statuses of {empty, in_progress, awaiting_user, complete, skipped, stale}. Three behaviours follow:

Enter anywhere. If a user has an existing design, he/she can begin at build, and if he/she has an existing workspace, he/she can begin at test.

Skip. Any stage can be skipped and each downstream handler must be able to accept missing artifacts from upstream handlers. No requirements are present when generating a test: warnings are issued and the test generation offers to capture the requirements or to infer a minimal set; it does not refuse.

Jump back and improve. If a stage is completed, re-entering it starts a new conversation round, outputs are new versions of artifacts, and any downstream stages are marked stale (not deleted) and remain inspectable and re-runnable.

Transition legality is the responsibility of the conductor, not the UI. It takes in an intent {message, target_stage, action ∈ refine | proceed | skip} and passes it to a stage handler, stores messages and artifacts, and sends out events. Centralising legality in one component is what keeps non-linearity a property of the system, and not one screen.

Design is before requirements in the product order because requirements should not be designed first, then the UI; if there is a specification, it is incorporated into the design provider's prompt as an additional context. This is a soft nudge in the interface, not a gate, as in the human-control evidence in [7,8].

IV. Requirements-Driven Verification

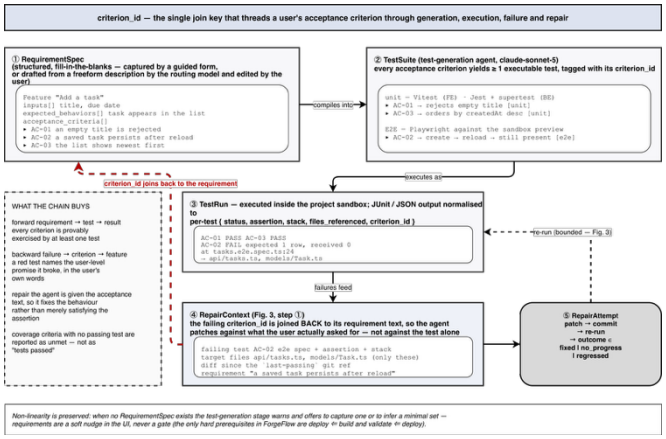


Fig. 3. The traceability chain. A user written acceptance criterion is passed through a single criterion_id which are used to generate and execute tests, as well as to analyse failures and repair them, and are reattached to the requirement text when attempting a patch.

A. Structured Capture

A RequirementSpec contains a list of features, all of which have a name, description, set of inputs, expected behaviours, and acceptance criteria. Capture is not free prose, but is deliberately structured to get deterministic, evaluable test generation from it. They have two entry points: one guided and the other is an AI draft where a freeform description is translated into a proposed feature list by the routing model. The draft is never auto-saved, it is presented for editing and for following the human-control principle.

B. Compilation to Executable Tests

The test-generation agent assigns each acceptance criterion to one or more executable test, each test being assigned with the identifier of the criterion that it tests: Vitest and Testing Library for frontend units, Jest with supertest for backend units, Playwright for end-to-end behaviour exercised against the sandbox preview. This is very similar to the E2E derivation in [18] with the exception that an explicit identifier is added for traceability to persist in execution.

C. Execution and Normalisation

Suites run in the project sandbox. The output of the JUnit and JSON reporters are normalised to a per-test record {status, assertion, stack, files_referenced, criterion_id}. Three reasons for normalisation:

1) The repair context in Section V is compiled deterministically from it.

2) A criterion that fails will not be included in an aggregate "tests passed" but will be reported as unmet.

3) The shape produced by the live validation run in Section VI is the same, so that the repair path is framework agnostic.

V. The Bounded, Diff-Aware Self-Healing Repair Loop

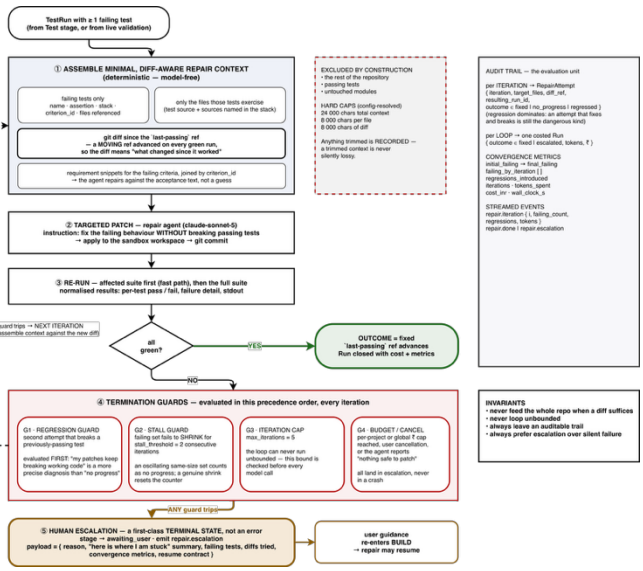


Fig. 4. The bounded self-healing repair loop: minimal context assembly, targeted patching, re-execution, four termination guards, in order of precedence, and escalation as terminal state.

This is the core contribution. It's given at least one TestRun that fails, the current workspace, and the requirements that the failing tests represent.

A. Minimal, Diff-Aware Repair Context

The context assembly is deterministic, model free and is exhaustively testable. It allows only four things to be admitted:

1) Only the failing tests are listed here: name, framework, assertion, stack, referenced source files, and criterion_id.

2) Only sources for those tests that fail: the failing test's source and files listed in the failing test's stack.

3) The difference between the current and last-passed line. This reference is moved: it is moved forward on each green run, not on the last commit, so the diff says "what changed since it worked", rather than "what changed in the last commit." After observing that tests generated by this approach are not robust with respect to evolution and that models do not attend to presented code, this is the mechanism that [11] recommends.

4) The requirement snippets for the failing criteria and a criterion_id, to allow the agent to repair against the acceptance text instead of just the assertion.

By construction, everything else is excluded (the rest of the repository, passing the tests, untouched modules). This result is capped (24 000 characters total, 8 000 per file, 8 000 of diff, all run-time-configurable); parts that are too large are simply dropped, and the number of times they are dropped is recorded, so a context which is too large is never silently lossy.

B. Targeted Patch and Re-execution

The repair agent only gets this context, and a hard constraint to repair the failing behaviour without breaking the passing tests. The patch is then put into the sandbox workspace and committed. The affected suite is rerun on the fast path and

then rerun on the full path to check for regressions the fast path may not have discovered.

C. The Loop Controller (Bounded Repair Controller)

Four guards terminate the loop, evaluated in a deliberate precedence order:

- **Regression guard.** Tripped on 2nd attempt when previously passing a test. It is evaluated first because "my patches keep breaking working code" is a more precise, and more actionable diagnosis than no progress. A single regression is accepted, as another legitimate patch can temporarily screw up one of the tests.
- **Stall guard.** If the failing-test set doesn't shrink within stall_threshold number of iterations, it is tripped. If the set is the same size, it counts as a no go, but if it shrinks, the counter resets.
- **Cap.** on number of iterations max_iterations, default 5, before each model call. The default is settled slightly above the one-to-three-iteration range where the literature suggests the bulk of the achievable gain is [9] and [10] and is known to be unproductive and potentially damaging [5] if iterated at the same time as the headroom is in the iteration zone.
- **Budget, Cancel no safe target**. Including crashing while the agent reports nothing safe to patch while the budget-cap is exhausted or the user cancels out.

Per attempt, RepairAttempt.outcome ∈ {fixed, no_progress, regressed} with regression being overwhelming: Attempt that fixes and breaks is still considered dangerous.

D. Escalation as a Terminal State

Each non-green ending resets the test phase to awaiting_user and sends an event of type repair.escalation with the reason field containing the reason, a short summary of the current state "here is where I am stuck", a set of failed tests, diffs already tried, the convergence metrics, and an explicit resume contract containing the name of the stage to resume guidance when it returns (Build). There is no error path to escalation. This is its encoded version of the invariant always prefer escalation over silent failure, and it is how the human-control pattern defined in [21] is extended to the repair phase.

E. Audit Trail and Convergence Metrics

The whole iteration persists as one versioned RepairAttempt that contains the target files, the diff reference, and the resulting run, while each iteration is persisted as a RepairAttempt with its target files, diff reference and resulting run.Each iteration is persisted as a RepairAttempt containing the target files, the diff reference, and the resulting run, and the whole iteration is persisted as one costed Run with the outcome {fixed, escalated}; a versioned repair report captures initial_failing, final_failing, failing_by_iteration[], regressions_introduced, iterations, tokens_spent, cost_inr and wall_clock_s. In the UI, repair.iteration {i, failing_count, regressions, tokens} events are sent over the loop, letting you see what failures occurred while the loop was running, and what did not. This is also the unit that the evaluation harness consumes (Section VIII).

The loop's invariants are: If a diff will do, do not feed the whole repository; If the loop will not terminate, bail out; Always leave an audible trail; Always prefer escalation to silent failure.

VI. Deployment and Live Validation

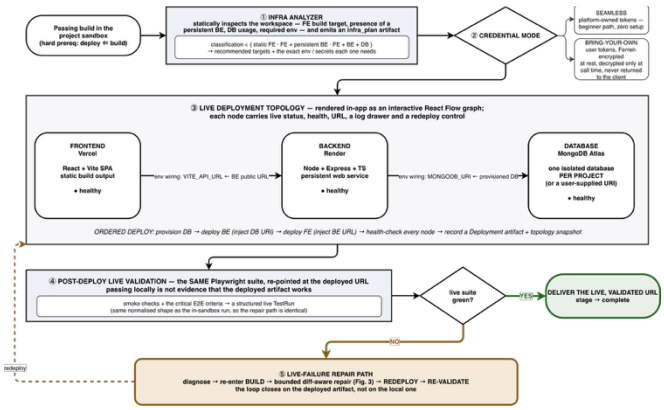


Fig. 5. The infrastructure analysis, dual mode credential handling, topology for deployment, and the path for live failure repair and validation after deployment.

A. Infrastrcuture Analysis

An analyser will analyse the workspace; frontend build target, whether or not a persistent backend will be attached, whether or not a database will be used and what type, required environment, etc.; and produce an infra_plan artefact that identifies the project as a static frontend, or a frontend with a persistent backend, or a frontend with a backend and a database, with recommended build targets and exact environment variables and secrets each needs. The motivation given in [6] is a 30.2 % / 26.8 % first-attempt deployment success rate; deployment configuration is a failure mode beyond the scope of generation quality.

B. Dual-Mode Credentials and Ordered Deployment

There are two ways to deploy: Seamless, which is deployed on the provider account that hosts the platform; or Bring Your Own, which is deployed with the user's own tokens. All provider tokens are encrypted at rest using Fernet, and will only be decrypted at call time, in process, and are never logged and never returned to the client. The frontend is deployed as a static SPA to Vercel and the backend to Render as a persistent Express process (a platform that hosts a persistent process that runs Express and a database cannot be collapsed onto a static first host). Order the deployment: provision the database; deploy the backend with the database URI injected; deploy the frontend with the public URL of the backend injected; health-check all nodes.Order the deployment: provision the database; deploy the backend with the database URI injected; deploy the frontend with the public URL of the backend injected; health-check all nodes. The resulting topology is depicted as an interactive React Flow graph with per-node status, logs and redeploy controls.

C. Post-Deployment Validation

It is then re-pointed to the deployed URL with the same Playwright suite. As the live run yields the same normalised shape as in-sandbox, upon failure, the live run re-enters the same repair loop as the in-sandbox run, which is Section V: diagnose, repair, redeploy, re-validate. This completes the circle, as the user will now have what they need for the artifact that they will use - the step which at present neither research systems nor commercial builders can do.

VII. Implementation

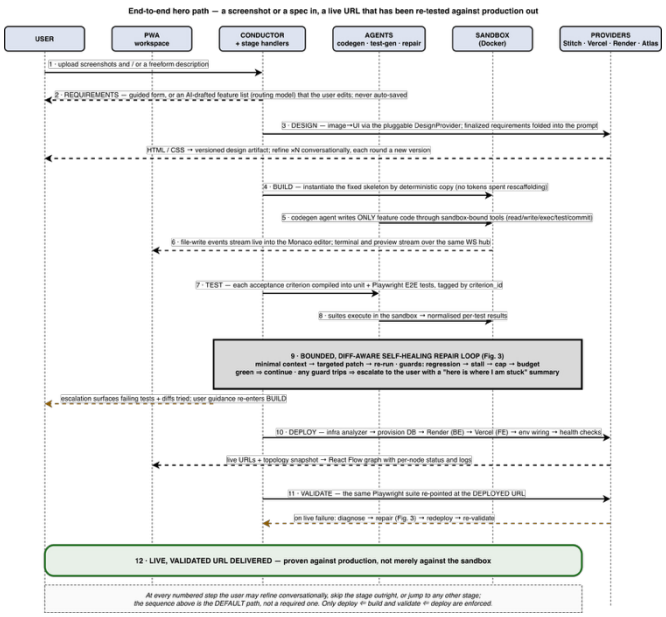


Fig. 6. The end-to-end hero path, from screenshot or specification intake to a live URL validated against production. Every numbered step remains refinable, skippable and re-enterable.

A complete prototype has been implemented across 53 independently shippable phases grouped into 12 epics, each phase gated on its own acceptance criteria and tests before being marked done. Table II summarises its scale.

TABLE II. Implementation Scale of the ForgeFlow Prototype

Component	Modules	Lines of code	Automated tests
Control plane (FastAPI, Python 3.11)	151	24 329	1 138 test functions / 163 files
Frontend PWA (React 18 + TypeScript)	128	16 502	353 test cases / 54 files
Generated-app skeleton (fixed stack)	44 files	—	placeholder suites (FE/BE/E2E)
Total production code	**279**	**40 831**	**1 491 automated tests**
Test code	217 files	32 020	—
Benchmark specifications	10 YAML specs	—	—

Notable Implementation Properties:

- **Layered configuration.** More than 100 runtime settings resolve with precedence; they are admin panel > environment > default and include the repair caps and the stall thresholds; the behaviour of the loop can thus be adjusted from anadministrative dashboard and tested by the evaluation harness without redeployment. At the interface, it's write-only as well, and it's encrypted at rest with Fernet encryption - the encryption key and the control-plane database URI are still in the environment, since they're read before the configuration layer exists and protect the ciphertext itself.
- **Quality gates.** TypeScript is strict with ESLint and Prettier rules; The Python codebaseruff, black andmypy clean withfull type hints and no bare exception handling. Errors are grouped into a single taxonomy that distinguishes between user and system and provider errors and the provider degradation error noticeably design-provider quota exhaustion is treated softly with an actionable message and a fallback, while no longer terminating the pipeline.
- **Security posture.** All authorisation is enforced per route; nocross-project data access is available; generated code never runs in the control plane; the sandbox doesn't have a host network and has dropcapabilities. This is done on purpose because of the credential- and error-handling smell classes reported for AI-generated infrastructurecode in [4] and the execution-environment vulnerabilities enumerated in [24] are located in the platform, not the prompt.

VIII. Discussion, Evaluation Protocol and Limitations

A. Discussion

The design of ForgeFlow can be interpreted as a series of engineering assurances based on the literature search of the last year that yielded negative results. The iterative self-improvement can compromise security [5] and little improvement will happen after a few rounds [9] [10] so there is a cap and a guard on the loop. By construction, generated tests are diff-first because they become invalid as soon as the code changes, and the suggested patch is a highlighted diff [11]. The pipeline is static and the skeleton is duplicated instead of generated because elaborate agency underperforms structured pipelines at higher cost [13]. The professionals stay in control, agents move away from intent at scale [7,8] and each stage is thus skip-able, redo-able and versionable, and non-convergence escalates to a human instead of eating budget. As quality of Generations cannot be considered a predictor of deployment success [6] deployment is followed by explicit classification of infrastructure and then validation with the live artifact.

B. Planned Evaluation Protocol

A headless evaluation harness is implemented and a benchmark context of ten application specifications is seeded with intake material, requirements and expected features. The harness runs the entire pipeline without the user interface and records, according to the requirements: 1st pass rate, passed

after repair, repair delta, repair iterations, regressions added, tokens and rupee cost, stage level and total wall-clock time, and a screen shot-to-URL runtime for running runs. The three arms of the intended study are: (i) ino repair, (ii) bounded diff-aware repair, which is the proposed configuration; and (iii) the bounded diff-aware repair with minimal diff-aware context ablated by removing the contribution of iteration. Secondary sweeps will use max_iterations and stall_threshold to find the empirical knee to justify and/or adjust the default values of 5 and 2, and to determine if the 1-3 iteration region reported in [9] and [10] holds true on whole-application generation rather than function-level benchmarks. In this paper, we do not report any quantitative results. Only infrastructure-level dry runs of the harness have been performed, no scored corpus run has been performed. As such, all the statements made above are statements about the design and implementation of ForgeFlow, and not about what it actually measures. The next task is to write a report on the study.

C. Threats to Validity

Construct. Pass rate on generated tests is computed based on agreement of the generated code to the generated tests, not correctness of generated tests to the user intent, the criterion_id chain mitigates but doesn't eliminate this, because a test can meet a criterion's letter but not its intent. **Internal.** The difference between modelnon-determinism and single runs is that the repeated runs of the same specification can vary, but the planned study is repeated runs with reported variance. The configuration layer is common to the evaluation harness and the system under test, so a sweepmust should be checked for not interfering with other behavior in the system. **External.** The results will be specific to a single generated-app stack, a model family and small/mid-sized CRUD-oriented web apps. They should not be generalized to other stacks, large existing codebases or repository-level maintenance tasks like those addressed in [12],[14]. **Conclusion.** The small corpus (ten specifications) means that effects of ablation may be too small to reach significance without expansion and repetition.

D. Limitations

As designed, the stack of the generated-app is fixed and restricts applicability. Design intake takes on the visual-fidelity limitations reported on in [23] which is why the stage is human-refinable. One provider per tier for deployment. The platform is for the creation of traditional Web applications and not mobile, desktop or native targets. Explicitly Outside the scope are: multi-user collaboration, payments in generated applications and enterprise compliance.

IX. Conclusion and Future Work

Verification is no longer the limiting factor in AI-powered software delivery, it's generation. 5 independent approaches to this are found in the literature from 2025 to 2026: AI-generated artifacts contain systematic defects in quality and security; iterative self-improvement can actually make the code worse; deployment may not work as expected; generated tests may be worse during code changes; developers do not want to give up control. ForgeFlow is a full, realized answer; a human-in-the loop, non-linear platform, which takes an idea from screenshots or a specification to a live URL, and has a limited, diff-aware, escalating repair loop in its center. It distinguishes itself with the negative commitment, instead of the positive one: the system will know when to stop trying, and will be able to return the problem with an auditable justification of what the system tried. Immediate future work is to perform the three-arm evaluation from Section VIII-B and report on the convergence, cost per success and latency (with variance); to add 20–30 additional specifications with increasing difficulty; to replace the fixed-loop iterations with a learned stopping criteria based on the observed failing-set trajectory; and check if the bounded-loop controller transfers to repository-level maintenance tasks, where the diff signal is richer, but the blast radius of a regression is considerably larger.

References

[1] Q. Zhang, C. Fang, Y. Xie, Y. Zhang, S. Yu, W. Sun, Y. Yang, and Z. Chen, "A survey on large language models for software engineering," Sci. China Inf. Sci., vol. 69, no. 4, Art. no. 141102, Mar. 2026, doi: 10.1007/s11432-025-4670-0.

[2] Y. Ge, L. Mei, Z. Duan, T. Li, Y. Zheng, Y. Wang, L. Wang, J. Yao, T. Liu, Y. Cai, B. Bi, F. Guo, J. Guo, S. Liu, and X. Cheng, "A survey of vibe coding with large language models," arXiv preprint arXiv:2510.12399, Oct. 2025.

[3] Stack Overflow, "2025 Developer Survey," Stack Overflow, 2025. Accessed: Aug. 16, 2026. [Online]. Available: https://survey.stackoverflow.co/2025/

[4] A. Ghammam and M. Almukhtar, "AI builds, we analyze: An empirical study of AI-generated build code quality," in Proc. 23rd Int. Conf. Mining Softw. Repositories (MSR), Rio de Janeiro, Brazil, 2026, pp. 727–731, doi: 10.1145/3793302.3793563.

[5] S. Shukla, H. Joshi, and R. Syed, "Security degradation in iterative AI code generation: A systematic analysis of the paradox," in Proc. 2025 IEEE Int. Symp. Technol. Soc. (ISTAS), Santa Clara, CA, USA, 2025, pp. 1–8, doi: 10.1109/ISTAS65609.2025.11269659.

[6] T. Zhang, S. Pan, Z. Zhang, Z. Xing, and X. Sun, "Deployability-centric infrastructure-as-code generation: An LLM-based iterative framework," arXiv preprint arXiv:2506.05623, Jun. 2025.

[7] N. Tang, C. Chen, G. Xu, Y. Shi, Y. Huang, C. McMillan, T. Dong, and T. J.-J. Li, "How coding agents fail their users: A large-scale analysis of developer-agent misalignment in 20,574 real-world sessions," arXiv preprint arXiv:2605.29442, May 2026.

[8] R. Huang, A. Reyna, S. Lerner, H. Xia, and B. Hempel, "Professional software developers don't vibe, they control: AI agent use for coding in 2025," arXiv preprint arXiv:2512.14012, Dec. 2025.

[9] T. Kiecker, E. Reichmann, H. Kang, G. An, and L. Grunske, "Is three the magic number? An empirical evaluation of LLM-based repair loops," arXiv preprint arXiv:2607.05197, Jul. 2026.

[10] J. J. Arimbur, "How many tries does it take? Iterative self-repair in LLM code generation across model scales and benchmarks," arXiv preprint arXiv:2604.10508, Apr. 2026.

[11] S. Haroon, M. T. Khan, and M. A. Gulzar, "Evaluating LLM-based test generation under software evolution," *arXiv preprint arXiv:2603.23443*, Mar. 2026.

[12] Z. Jiang, D. Lo, and Z. Liu, "Agentic software issue resolution with large language models: A survey," *arXiv preprint arXiv:2512.22256*, Dec. 2025.

[13] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-based software engineering agents," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 801–824, 2025, doi: 10.1145/3715754.

[14] I. Bouzenia, P. T. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Ottawa, ON, Canada, 2025, pp. 2188–2200, doi: 10.1109/ICSE55347.2025.00157.

[15] K. Huang, J. Zhang, X. Bao, X. Wang, and Y. Liu, "Comprehensive fine-tuning large language models of code for automated program repair,"

IEEE Trans. Softw. Eng., vol. 51, no. 4, pp. 904–928, Apr. 2025, doi: 10.1109/TSE.2025.3532759.

[16] W. C. Ouédraogo, A. K. Kaboré, Y. Li, H. Tian, A. Koyuncu, J. Klein, D. Lo, and T. F. Bissyandé, "Prompt engineering in LLMs for automated unit test generation: A large-scale study," *Empirical Softw. Eng.*, vol. 31, no. 4, Art. no. 103, 2026, doi: 10.1007/s10664-026-10840-4.

[17] Y. Shang, Q. Zhang, C. Fang, S. Gu, J. Zhou, and Z. Chen, "A large-scale empirical study on fine-tuning large language models for unit testing," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, Art. no. ISSTA074, pp. 1678–1700, 2025, doi: 10.1145/3728951.

[18] P. Alian, N. Nashid, M. Shahbandeh, T. Shabani, and A. Mesbah, "Feature-driven end-to-end test generation," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Ottawa, ON, Canada, 2025, pp. 450–462, doi: 10.1109/ICSE55347.2025.00141.

[19] S. Gu, N. Nashid, and A. Mesbah, "LLM test generation via iterative hybrid program analysis," *arXiv preprint arXiv:2503.13580*, Mar. 2025.

[20] N. Alturayeif, I. Ahmad, and J. Hassine, "TraceLLM: Leveraging large language models with prompt engineering for enhanced requirements traceability," *Requirements Eng.*, vol. 31, no. 1, Art. no. 6, 2026, doi: 10.1007/s00766-026-00460-1.

[21] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng.: Softw. Eng. in Practice (ICSE-SEIP)*, Ottawa, ON, Canada, 2025, pp. 342–352, doi: 10.1109/ICSE-SEIP66354.2025.00036.

[22] J. Pasuksmit, W. Takerngsaksiri, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, S. Wang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents: Challenges and future directions," in *Proc. IEEE/ACM 22nd Int. Conf. Mining Softw. Repositories (MSR)*, Ottawa, ON, Canada, 2025, pp. 756–757, doi: 10.1109/MSR66628.2025.00112.

[23] C. Si, Y. Zhang, R. Li, Z. Yang, R. Liu, and D. Yang, "Design2Code: Benchmarking multimodal code generation for automated front-end engineering," in *Proc. 2025 Conf. Nations Americas Chapter Assoc. Comput. Linguist.: Human Lang. Technol. (NAACL)*, Albuquerque, NM, USA, 2025, pp. 3956–3974, doi: 10.18653/v1/2025.naacl-long.199.

[24] R. Rabin, J. Hostetler, S. McGregor, B. Weir, and J. Judd, "SandboxEval: Towards securing test environment for untrusted code," *arXiv preprint arXiv:2504.00018*, Mar. 2025.